

Concorrência Deadlock e Starvation

Sistemas Operativos

2024/2025

Baseado nos slides do **Capítulo 6 (Concurrency: Deadlock and Starvation)** do livro Operating Systems: Internals and Design Principles, 7th Edition, Dave Bremer, Otago Polytechnic, N.Z., 2008, Prentice Hall.

Deadlock e Starvation

“When two trains approach each other at a crossing, both shall come to a full stop and neither shall start up again until the other has gone. Statute passed by the Kansas”

State Legislature, early in the 20th century.

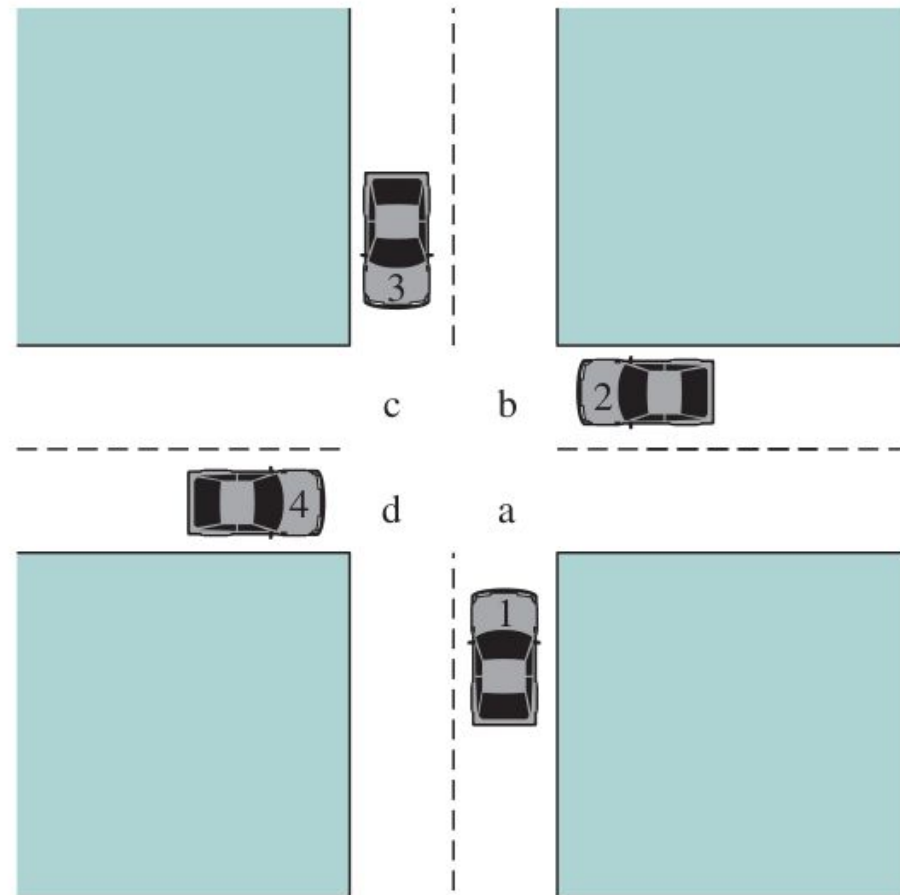
A TREASURY OF RAILROAD FOLKLORE,
B. A. Botkin and Alvin F. Harlow

Deadlock

- Bloqueio permanente de um conjunto de processos, que competem por recursos do sistema, ou comunicam entre si
- Um conjunto de processos está em deadlock quando cada processo no conjunto está bloqueado, à espera de um evento, que apenas pode ser despoletado por outro processo bloqueado no mesmo conjunto de processos.
- Permanente
- Não existe uma solução ideal

Potencial Deadlock

- Carro 1:
 - Precisa de quadrante **a** e **b**
- Carro 2:
 - Precisa de quadrante **b** e **c**
- Carro 3:
 - Precisa de quadrante **c** e **d**
- Carro 4:
 - Precisa de quadrante **d** e **a**

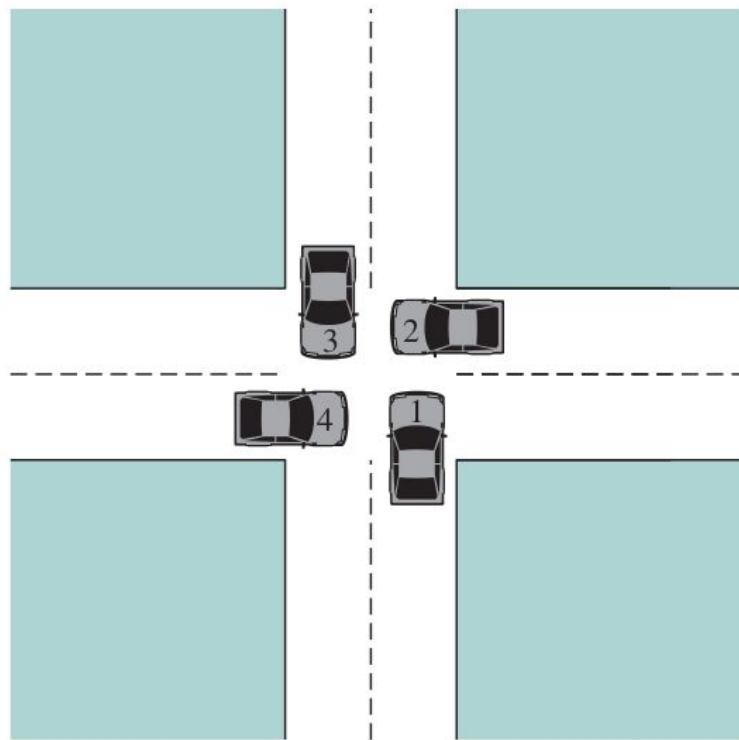


(a) Deadlock possible

Figure 6.1 Illustration of Deadlock

Deadlock

- Carro 1:
 - Precisa de quadrante **a** e **b**
- Carro 2:
 - Precisa de quadrante **b** e **c**
- Carro 3:
 - Precisa de quadrante **c** e **d**
- Carro 4:
 - Precisa de quadrante **d** e **a**



(b) Deadlock

Figure 6.1 Illustration of Deadlock

Exemplo: Recursos D e T

Step	Process P Action
p ₀	Request (D)
p ₁	Lock (D)
p ₂	Request (T)
p ₃	Lock (T)
p ₄	Perform function
p ₅	Unlock (D)
p ₆	Unlock (T)

Step	Process Q Action
q ₀	Request (T)
q ₁	Lock (T)
q ₂	Request (D)
q ₃	Lock (D)
q ₄	Perform function
q ₅	Unlock (T)
q ₆	Unlock (D)

Figure 6.4 Example of Two Processes Competing for Reusable Resources

Outro exemplo

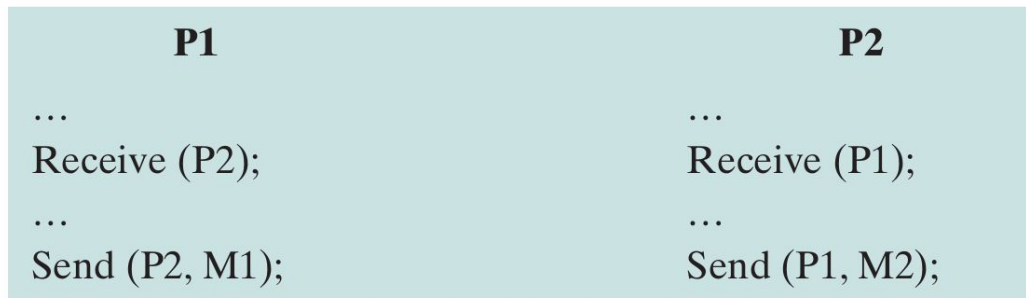
- Assumindo que apenas existem 200 Kbytes disponíveis para alocação, e acontece a seguinte sequência de eventos

P1	P2
...	...
Request 80 Kbytes;	Request 70 Kbytes;
...	...
Request 60 Kbytes;	Request 80 Kbytes;

- Acontece deadlock se ambos os processos avançarem para o 2º pedido de alocação de memória

Recursos consumíveis

- Considerem-se 2 processos, onde cada processo tenta receber uma mensagem do outro e depois envia uma mensagem ao outro processo



- Acontece Deadlock **se o Receive for bloqueante**

Abordagens

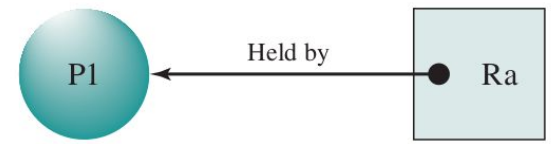
Table 6.1 Summary of Deadlock Detection, Prevention, and Avoidance Approaches for Operating Systems [ISLO80]

Approach	Resource Allocation Policy	Different Schemes	Major Advantages	Major Disadvantages
Prevention	Conservative; undercommits resources	Requesting all resources at once	• Works well for processes that perform a single burst of activity • No preemption necessary	• Inefficient • Delays process initiation • Future resource requirements must be known by processes
		Preemption	• Convenient when applied to resources whose state can be saved and restored easily	• Preempts more often than necessary
		Resource ordering	• Feasible to enforce via compile-time checks • Needs no run-time computation since problem is solved in system design	• Disallows incremental resource requests
Avoidance	Midway between that of detection and prevention	Manipulate to find at least one safe path	• No preemption necessary	• Future resource requirements must be known by OS • Processes can be blocked for long periods
Detection	Very liberal; requested resources are granted where possible	Invoke periodically to test for deadlock	• Never delays process initiation • Facilitates online handling	• Inherent preemption losses

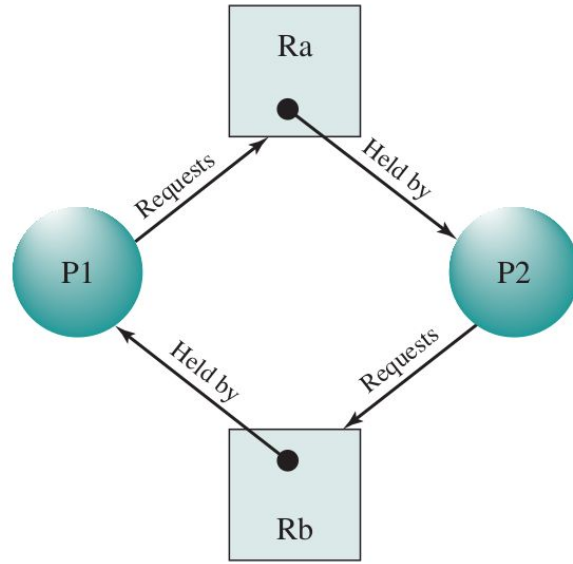
Grafos de alocação de recursos



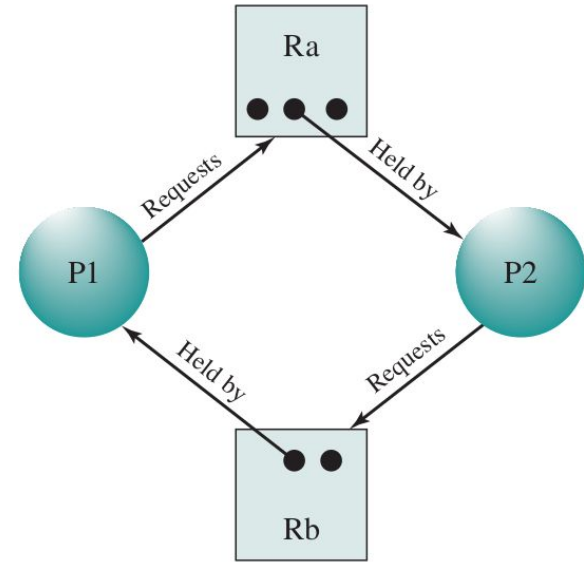
(a) Resource is requested



(b) Resource is held



(c) Circular wait



(d) No deadlock

Figure 6.5 Examples of Resource Allocation Graphs

Grafos de alocação de recursos

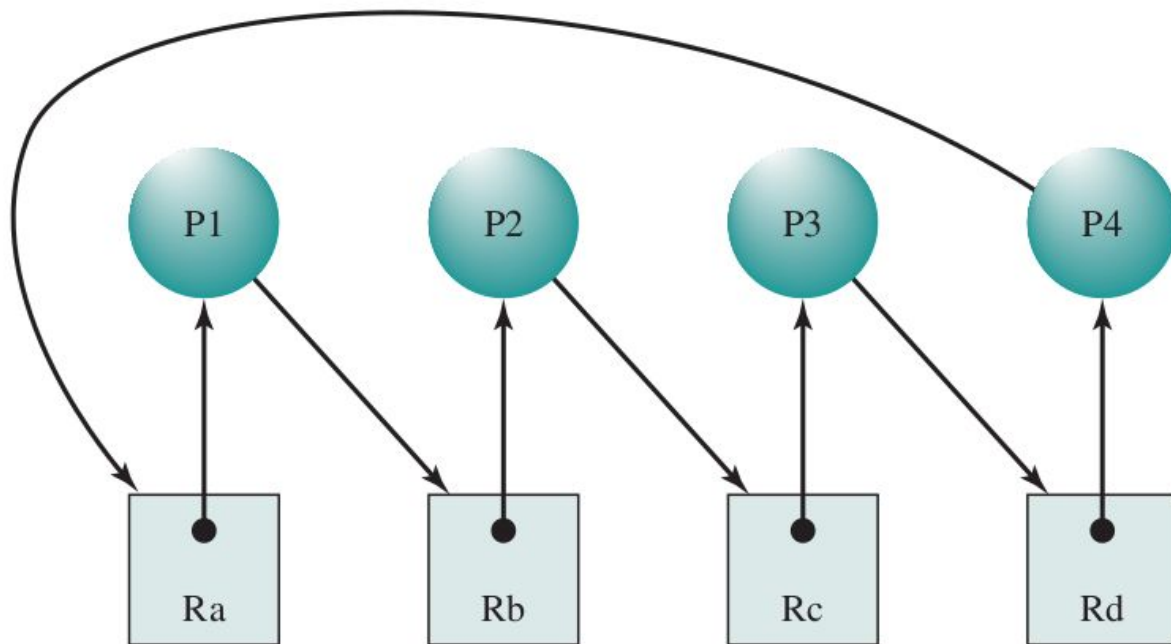


Figure 6.6 Resource Allocation Graph for Figure 6.1b

Condições para Deadlock

1. Exclusão mútua

- apenas um processo pode usar um recurso de cada vez

2. Hold-and-wait

- um processo pode alocar recursos enquanto está à espera que lhe sejam atribuídos mais recursos

3. Não existência de *preemption*

- os recursos não podem ser removidos de um processo que está a bloqueá-lo

4. Circular wait

- existência de uma cadeia circular (fechada) de processos, onde cada processo bloqueia pelo menos um recurso necessário pelo próximo processo na cadeia

Estratégia de prevenção de Deadlock

- *Deadlock Prevention*
- Desenhar um sistema de tal forma que não permita deadlocks
- Duas abordagens
 - Indireta
 - prevenir a ocorrência de uma das 3 condições necessárias (itens 1 a 3)
 - Direta
 - prevenir a ocorrência de um “circular wait” (item 4)

Condição para prevenção de deadlocks

- Prevenir a Exclusão mútua não é permitido
 - se o acesso a um recurso precisar de exclusão mútua, então deve ser suportado pelo sistema operativo
- Prevenir Hold and Wait
 - Obriga a que um processo solicite todos os recursos que precisa, de uma só vez, bloqueando o processo até que todos os recursos sejam disponibilizados, todos de uma vez
- Preemption
 - se um processo que contém determinados recursos tiver um pedido adicional recusado, esse processo deverá liberar os seus recursos originais
 - ou, se um processo pedir um recurso adicional que está bloqueado, o SO força o processo que está a bloquear o recurso a libertá-lo
- Prevenir Circular Wait
 - Definir uma ordem linear de tipo de recursos
 - Se forem alocados recursos do tipo R a um processo, ele poderá apenas pedir recursos dos tipos seguintes a R na ordenação.

Evitar deadlocks (Deadlock avoidance)

- Mais eficiente na alocação de recursos e na execução de processos que prevenir deadlocks
- Quando um pedido para a alocação de um recurso é feito, se o recurso for alocado, é feita uma decisão de forma dinâmica se essa alocação pode resultar num deadlock
- Obriga a **ter conhecimento de pedidos futuros** de alocação de recursos

Negação de alocação de recursos

- Conhecido como algoritmo do banqueiro
- Estado (State) do sistema
 - reflete a alocação atual de recursos a processos
- Estado seguro (Safe state)
 - estado em que existe pelo menos uma sequência de alocação de recursos a processos que não resulta num deadlock
- Estado não seguro (Unsafe state)
 - estado que não é seguro

Determinação de um Estado Seguro

- Algoritmo:
 - *Pode algum processo correr até completar a partir deste estado?*
 - Responder até “não haver processos a correr”
- Exemplo:
 - Estado de um sistema com 4 processos e 3 recursos
 - Foram feitas as alocações aos 4 processos

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Claim matrix C

	R1	R2	R3
P1	1	0	0
P2	6	1	2
P3	2	1	1
P4	0	0	2

Allocation matrix A

	R1	R2	R3
P1	2	2	2
P2	0	0	1
P3	1	0	3
P4	4	2	0

C – A

R1	R2	R3
9	3	6

Resource vector R

R1	R2	R3
0	1	1

Available vector V

(a) Initial state

P2 corre até ao fim

	R1	R2	R3
P1	3	2	2
P2	0	0	0
P3	3	1	4
P4	4	2	2

Claim matrix C

	R1	R2	R3
P1	1	0	0
P2	0	0	0
P3	2	1	1
P4	0	0	2

Allocation matrix A

	R1	R2	R3
P1	2	2	2
P2	0	0	0
P3	1	0	3
P4	4	2	0

C - A

R1	R2	R3
9	3	6

Resource vector R

R1	R2	R3
6	2	3

Available vector V

(b) P2 runs to completion

P1 corre até ao fim

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	3	1	4
P4	4	2	2

Claim matrix C

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	2	1	1
P4	0	0	2

Allocation matrix A

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	1	0	3
P4	4	2	0

$C - A$

R1	R2	R3
9	3	6

Resource vector R

R1	R2	R3
7	2	3

Available vector V

(c) P1 runs to completion

P3 corre até ao fim

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	4	2	2

Claim matrix C

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	0	0	2

Allocation matrix A

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	4	2	0

C - A

R1	R2	R3
9	3	6

Resource vector R

R1	R2	R3
9	3	4

Available vector V

(d) P3 runs to completion

Determinar um estado não seguro

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Claim matrix C

	R1	R2	R3
P1	1	0	0
P2	5	1	1
P3	2	1	1
P4	0	0	2

Allocation matrix A

	R1	R2	R3
P1	2	2	2
P2	1	0	2
P3	1	0	3
P4	4	2	0

$C - A$

R1	R2	R3
9	3	6

Resource vector R

R1	R2	R3
1	1	2

Available vector V

(a) Initial state

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Claim matrix C

	R1	R2	R3
P1	2	0	1
P2	5	1	1
P3	2	1	1
P4	0	0	2

Allocation matrix A

	R1	R2	R3
P1	1	2	1
P2	1	0	2
P3	1	0	3
P4	4	2	0

$C - A$

R1	R2	R3
9	3	6

Resource vector R

R1	R2	R3
0	1	1

Available vector V

(b) P1 requests one unit each of R1 and R3

Figure 6.8 Determination of an Unsafe State

Evitar deadlocks (deadlock avoidance)

Vantagens

- Não é necessário fazer a “preemption” ou o “rollback” de processos
 - como acontece na detecção de deadlocks
- É menos restritivo do que a prevenção deadlocks

Evitar deadlocks (deadlock avoidance)

Restrições

- Deve saber-se o número máximo de recursos necessários a usar por cada processo
- Processos em consideração devem ser independentes e não terem quaisquer requisitos de sincronização
- Deve existir um número fixo de recursos para alocar
- Nenhum processo pode terminar enquanto está a bloquear recursos

Algoritmos de detecção de deadlocks

- A verificação de deadlocks pode ser
 - muito frequente: sempre que existe o pedido de um recurso
 - menos frequente: dependendo da probabilidade de um deadlock ocorrer
- Vantagens
 - deteção rápida de deadlocks
 - algoritmo relativamente simples
- Desvantagens
 - verificações frequentes consomem tempo considerável de processador

Algoritmo de detecção de deadlocks

	R1	R2	R3	R4	R5
P1	0	1	0	0	1
P2	0	0	1	0	1
P3	0	0	0	0	1
P4	1	0	1	0	1

Request matrix Q

	R1	R2	R3	R4	R5
P1	1	0	1	1	0
P2	1	1	0	0	0
P3	0	0	0	1	0
P4	0	0	0	0	0

Allocation matrix A

R1	R2	R3	R4	R5
2	1	1	2	1

Resource vector

R1	R2	R3	R4	R5
0	0	0	0	1

Available vector

Figure 6.10 Example for Deadlock Detection

Algoritmo de deadlocks

1. Marcar todos os processos que têm uma linha na matriz de **Alocação A só com zeros**
2. Inicializar um vetor temporário **W** igual ao vector **Available** que tem tamanho m.
3. Encontrar um **i**, tal que o processo **i** não se encontra marcado e a linha **i de Q é menor ou igual a W**. Ou seja $Q_{ik} \leq W_k$, para $1 \leq k \leq m$. Se tal linha não existir, o algoritmo termina.
4. Se tal linha for encontrada, marcar o processo **i** e adicionar a linha correspondente da matriz de alocação a **W**. Ou seja, $W_k = W_k + A_{ik}$, para $1 \leq k \leq m$. Voltar ao passo 3.

1. Abortar todos os processos deadlocked (das soluções mais comum)
 2. Fazer backup de cada um dos processos deadlocked para um checkpoint previamente definido e reiniciar todos os processos
 3. Sucessivamente abortar processos deadlocked, até que deixe de existir deadlock
 4. Fazer a preemption de recursos até que o deadlock deixe de existir
- Para 3 e 4 é necessário um critério de seleção do processo:
 - menos tempo de CPU consumido ou que falta terminar
 - menos outputs produzidos
 - menos recursos alocados
 - menos prioridade

Exemplo

- Programar com semáforos
- Problema clássico
 - **Dining Philosophers Problem**

Dining Philosophers Problem

- Não podem existir 2 filósofos a usarem o mesmo garfo ao mesmo tempo
 - exclusão mútua
- Nenhum filósofo pode morrer à fome
 - evitar deadlock e starvation

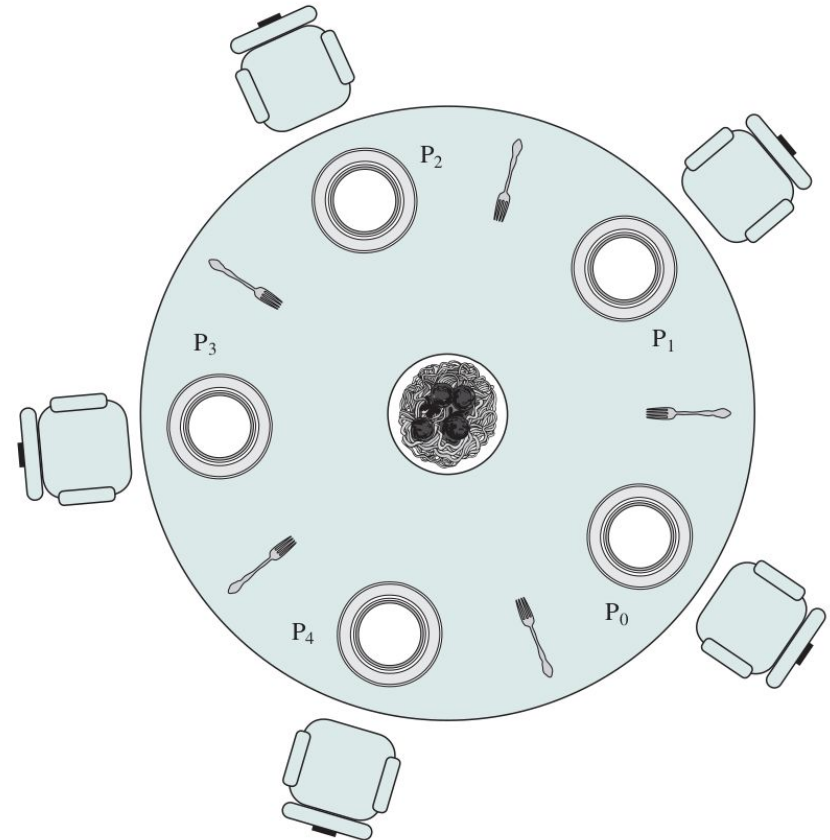


Figure 6.11 Dining Arrangement for Philosophers

Solução usando semáforos

```
/* program   diningphilosophers */
semaphore fork [5] = {1};
int i;
void philosopher (int i)
{
    while (true) {
        think();
        wait (fork[i]);
        wait (fork [(i+1) mod 5]);
        eat();
        signal(fork [(i+1) mod 5]);
        signal(fork[i]);
    }
}
void main()
{
    parbegin (philosopher (0), philosopher (1),
              philosopher (2),    philosopher (3),
              philosopher (4));
}
```

Figure 6.12 A First Solution to the Dining Philosophers Problem

Outra solução sem deadlocks e starvation

```
/* program   diningphilosophers */
semaphore fork[5] = {1};
semaphore room = {4};
int i;
void philosopher (int i)
{
    while (true) {
        think();
        wait (room);
        wait (fork[i]);
        wait (fork [(i+1) mod 5]);
        eat();
        signal (fork [(i+1) mod 5]);
        signal (fork[i]);
        signal (room);
    }
}
void main()
{
    parbegin (philosopher (0), philosopher (1),
              philosopher (2), philosopher (3),
              philosopher (4));
}
```

Figure 6.13 A Second Solution to the Dining Philosophers Problem

- Deadlock é o bloqueio de um conjunto de processos que competem por recursos do sistema ou que comunicam entre si
 - O bloqueio é permanente até o SO realizar uma determinada ação (ex: terminar um processo)
- Existem 3 tipos de abordagens para lidar com deadlocks
 - Prevenir Deadlocks (deadlock prevention)
 - Garante que não existem ocorrências de deadlocks ao evitar, através do *design*, que uma das condições aconteça
 - Evitar Deadlocks (Deadlock avoidance)
 - Precisa ter conhecimento que recursos vão ser necessários por processo
 - Analisa se o recurso que foi pedido pode resultar num deadlock
 - Detetar Deadlocks (Deadlock detection)
 - O SO verifica num determinado momento se existe um deadlock
 - Executa uma ação para terminar o deadlock (ex termina um dos processos)